

DISM

Entrega 2

-

**Gestión, Registro, Consulta
de Fichajes**

-

Santino Campessi Lojo

-

01/12/2025

Tabla de contenido

Introducción	4
API Endpoints (ServiciosRest)	5
Usuarios.....	5
Trabajos.....	5
Fichajes.....	5
ApiKey	6
Aplicación AppUsuarios.....	7
Inicio de Sesión	7
Página de Inicio.....	8
Página Registrar Fichaje	8
Estructura General.....	8
Carga de Datos	9
Obtención de Geolocalización y Mapa.....	9
Registro y Finalización de Fichajes.....	11
Limpieza y Control de Estado	12
Página Consultar Fichaje.....	13
Aplicación AppAdmin	14
Acceso Restringido	14
Gestión de Trabajos y Usuarios	14
Gestión de Fichajes y Mapas	15
Seguridad y Autenticación	16
Middleware de Seguridad (ApiKey)	16
Autenticación (Login)	16
Swagger (OpenApi)	17
Definición de Metadatos	17
Ejemplo Método HTTP (GET Usuarios).....	17
Components / Schema (Usuario).....	18
Interfaz Visual del Swagger	18
Manual de Usuario	19
Puesta en Marcha ServicioRest.....	19
Ejecución de la aplicación Ionic	20
Pruebas del Funcionamiento.....	21
AppUsuarios.....	21

Registro de un Fichaje	21
Consulta de Fichaje	23
AppAdmin.....	24
Pantalla Inicial tras Login	24
Gestión de Usuarios	24
Gestión de Trabajos	25
Gestión de Fichajes	26

Introducción

El objetivo de este proyecto es desarrollar un sistema de registro horario que permita a los trabajadores registrar sus fichajes y consultar sus horas trabajadas. En esta primera parte se han implementado la AppUsuarios y los Servicios REST, que se comunican con una base de datos MySQL.

La AppUsuarios, desarrollada con Ionic y Angular, permite al usuario iniciar y finalizar fichajes asociados a un trabajo, registrando la hora y la ubicación mediante los plugins de Geolocation y la librería Leaflet para mostrar el punto en un mapa. También ofrece la posibilidad de consultar los fichajes realizados filtrando por fechas.

Por su parte, los Servicios REST, creados con Node.js y Express, gestionan las peticiones de la app, realizando las operaciones necesarias sobre la base de datos y devolviendo la información en formato JSON. Esta estructura cliente-servidor permite separar la lógica de negocio de la interfaz, facilitando el mantenimiento y futuras ampliaciones.

API Endpoints (ServiciosRest)

Usuarios

Método	Endpoint	Parámetros	Descripción
GET	/usuarios		Devuelve el listado de usuarios.
GET	/usuarios/:id	id(URL)	Devuelve la información usuario.
POST	/usuarios	{ nombre, usuario, clave }	Crea un nuevo usuario
POST	/usuarios/login	{ usuario, clave }	Comprueba las credenciales para el login.
PUT	/usuarios/:id	{ nombre, usuario, clave }	Actualiza un usuario
DELETE	/usuarios/:id	id(URL)	Elimina un usuario

Trabajos

Método	Endpoint	Parámetros	Descripción
GET	/trabajos		Devuelve el listado de trabajos.
GET	/trabajos/:id	id(URL)	Devuelve la información del trabajo.
POST	/trabajos	{ nombre }	Crea un nuevo trabajo
PUT	/trabajos/:id	{ nombre }	Actualiza un trabajo
DELETE	/trabajos/:id	id(URL)	Elimina un trabajo

Fichajes

Método	Endpoint	Parámetros	Descripción
GET	/fichajes		Devuelve el listado de fichajes.
GET	/fichajes/usuario	usuariold, fechaInicio?, fechaFin?	Devuelve la información del fichaje.
GET	/fichajes/actual/:usuariold	usuariold (URL)	Devuelve la información del fichaje actual.
POST	/fichajes	{ IdUsuario, IdTrabajo, GeolocalizaciónLatitud, GeolocalizaciónLongitud }	Crea un nuevo fichaje.
PUT	/trabajos/:id/finalizar	Id(URL)	Finaliza un fichaje.

ApiKey

Método	Endpoint	Parámetros	Descripción
GET	/apikeys		Devuelve el listado de claves.
GET	/apikeys/:id	id(URL)	Devuelve la información de una clave.
GET	/apikeys/verificar/:clave	clave(URL)	Verifica si la clave existe.
POST	/apikeys	{ key }	Crea una nueva clave.
DELETE	/apikeys/:id	id(URL)	Elimina una clave.

Aplicación AppUsuarios

La aplicación AppUsuarios se ha desarrollado con el framework Ionic (Angular + Capacitor) y su objetivo es permitir a los empleados registrar y consultar sus fichajes de forma sencilla, desde un entorno web o móvil.

En esta última versión se han implementado cuatro páginas principales:

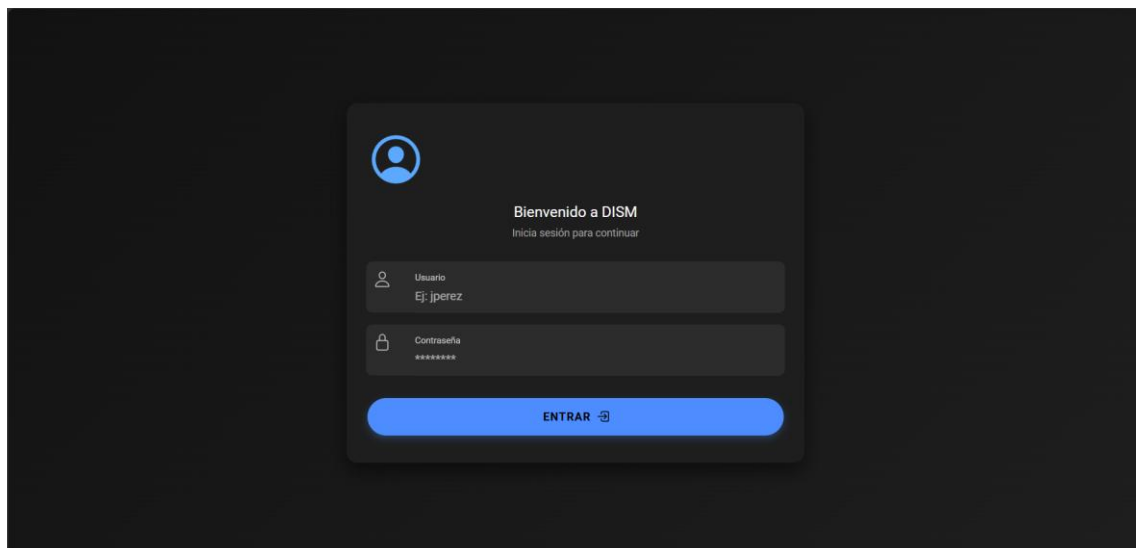
- Login: pantalla para el inicio de sesión
- Inicio: pantalla de bienvenida y navegación.
- Registrar fichaje: permite iniciar y finalizar fichajes, incluyendo geolocalización y mapa.
- Consultar fichajes: muestra un historial de registros filtrados por usuario y fecha.

Toda la aplicación se comunica con el backend mediante servicios REST en formato JSON. Las peticiones se gestionan con HttpClient y los datos se muestran usando componentes de Ionic.

Inicio de Sesión

Al abrir la aplicación, el usuario se encontrará con una nueva pantalla de Login. Tras introducir usuario y contraseña:

1. La App valida los datos contra el servidor.
2. Si son correctos, guardo la sesión localmente y doy acceso al menú principal.
3. Gracias a un Interceptor que he configurado, todas las peticiones que se hagan a partir de ahora incluirán automáticamente la ApiKey de seguridad.



Página de Inicio

La HomePage actúa como menú principal. Desde aquí el usuario puede acceder a las dos secciones clave: Registrar Fichaje y Consultar Fichajes.

La interfaz usa un ion-card con dos botones (ion-button) que redirigen mediante routerLink. El código TypeScript simplemente define el componente y su navegación.

Página Registrar Fichaje

Su propósito es que el trabajador pueda iniciar un fichaje, registrando usuario, trabajo, hora y ubicación exacta en el mapa.

También permite finalizar un fichaje activo, calculando automáticamente las horas trabajadas.

Estructura General

La interfaz está dividida en dos tarjetas (ion-card):

- Una para iniciar un nuevo fichaje, visible solo cuando no hay un fichaje activo.
- Otra para finalizar el fichaje activo, que se muestra cuando el sistema detecta que el usuario tiene uno abierto.

El control de esta lógica se realiza con la variable fichajeActivo en el componente TypeScript.

Carga de Datos

Al entrar en la página (`ionViewWillEnter()`), se cargan los usuarios y trabajos desde los endpoints `/usuarios` y `/trabajos`, usando los servicios `UsuariosService` y `TrabajosService`.

```
async ionViewWillEnter() {
  this.obtenerUsuarios();
  this.obtenerTrabajos();
}

obtenerUsuarios() {
  this.usuariosService.getUsuarios().subscribe({
    next: res => this.usuarios = res,
    error: err => console.error('Error cargando usuarios', err)
  });
}

obtenerTrabajos() {
  this.trabajosService.getTrabajos().subscribe({
    next: res => this.trabajos = res,
    error: err => console.error('Error cargando trabajos', err)
  });
}
```

Cuando el usuario selecciona su nombre, se ejecuta el método `comprobarFichaje()`, que consulta el endpoint `/fichajes/actual/:usuarioid`.

```
comprobarFichaje() {
  if (!this.usuarioSeleccionado) return;
  this.fichajesService.getFichajeActual(this.usuarioSeleccionado).subscribe({
    next: data => this.fichajeActivo = data,
    error: () => this.fichajeActivo = null
  });
}
```

Si existe un fichaje abierto, el sistema lo muestra directamente sin permitir crear uno nuevo.

Obtención de Geolocalización y Mapa

Al pulsar el botón “Obtener Geolocalización”, se ejecuta el método `locate()`. Este utiliza el plugin `@capacitor/geolocation` para obtener las coordenadas GPS y luego realiza una petición HTTP al servicio de OpenStreetMap (Nominatim) para obtener la dirección legible.

```
async locate() {
  const coordinates = await Geolocation.getCurrentPosition();
  this.latitud = coordinates.coords.latitude;
  this.longitud = coordinates.coords.longitude;

  const url = `https://nominatim.openstreetmap.org/reverse?format=json&lat=${this.latitud}&lon=${this.longitud}&addressdetails=1`;
  this.http.get(url).subscribe((data: any) => {
    this.direccionGeorreferenciada = data.display_name;
  });

  setTimeout(() => {
    this.cargarMapa();
  }, 100);
}
```

Además, contiene un pequeño temporizador para que el mapa tenga los datos necesarios para su carga. Una vez disponibles los datos, se renderiza un mapa de Leaflet centrado en la ubicación actual.

El mapa incluye un marcador personalizado que señala la posición y un popup informativo.

El método cargarMapa() crea el mapa dinámicamente y gestiona el caso de que ya exista uno (eliminándolo para evitar errores de duplicado).

```
cargarMapa() {
  if (this.map) {
    this.map.remove();
    this.map = undefined;
  }

  const iconRetinaUrl = 'assets/marker-icon-2x.png';
  const iconUrl = 'assets/marker-icon.png';
  const shadowUrl = 'assets/marker-shadow.png';
  const iconDefault = icon({
    iconRetinaUrl,
    iconUrl,
    shadowUrl,
    iconSize: [25, 41],
    iconAnchor: [12, 41],
    popupAnchor: [1, -34],
    tooltipAnchor: [16, -28],
    shadowSize: [41, 41],
  });
  Marker.prototype.options.icon = iconDefault;

  this.map = Leaflet.map('mapId').setView([this.latitud!, this.longitud!], 15);

  Leaflet.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
    attribution: '© OpenStreetMap contributors',
  }).addTo(this.map);

  Leaflet.marker([this.latitud!, this.longitud!])
    .addTo(this.map)
    .bindPopup('Ubicación actual')
    .openPopup();
}
```

Para garantizar la correcta visualización del mapa, es necesario incluir código CSS.

```
#mapId {
  height: 300px;
  width: 100%;
  border-radius: 10px;
}
```

Registro y Finalización de Fichajes

Una vez seleccionados el usuario, trabajo y ubicación, el botón “Iniciar Fichaje” ejecuta registrarEntrada().

Este método envía los datos al backend (POST /fichajes) y muestra una alerta de confirmación si la operación tiene éxito.

```
registrarEntrada() {
  if (!this.usuarioSeleccionado || !this.trabajoSeleccionado || !this.latitud || !this.longitud) {
    alert('Selecciona usuario, trabajo y asegúrate de tener geolocalización.');
```

```
    return;
  }

  const data = {
    IdUsuario: this.usuarioSeleccionado,
    IdTrabajo: this.trabajoSeleccionado,
    GeolocalizacionLatitud: this.latitud,
    GeolocalizacionLongitud: this.longitud,
  };

  this.fichajesService.createFichaje(data).subscribe(async res => {
    const alert = await this.alertCtrl.create({
      header: 'Fichaje registrado',
      message: res.message,
      buttons: ['OK'],
    });
    await alert.present();
    this.comprobarFichaje();

    this.limpiarCampos();
  });
}
```

Después, limpia los campos y vuelve a comprobar si el usuario tiene un fichaje activo.

Cuando el fichaje está en curso, se muestra la segunda tarjeta con la información del trabajo y la hora de entrada.

El botón “Finalizar Fichaje” ejecuta finalizarFichaje(), que llama al endpoint PUT /fichajes/:id/finalizar para cerrar el registro y calcular las horas trabajadas. También muestra un mensaje con las horas y limpia el formulario.

```
finalizarFichaje() {
  this.fichajesService.finalizarFichaje(this.fichajeActivo.IdFichaje).subscribe(async res => {
    const alert = await this.alertCtrl.create({
      header: 'Fichaje finalizado',
      message: `Horas trabajadas: ${res.horasTrabajadas}`,
      buttons: ['OK'],
    });
    await alert.present();
    this.fichajeActivo = null;
    this.limpiarCampos();
  });
}
```

Limpieza y Control de Estado

Para mantener la aplicación estable, se ha añadido un método limpiarCampos() que se encarga de resetear todos los datos (selecciones, mapa, coordenadas).

```
limpiarCampos() {  
  this.trabajoSeleccionado = undefined;  
  this.latitud = undefined;  
  this.longitud = undefined;  
  this.direccionGeorreferenciada = undefined;  
  
  if (this.map) {  
    this.map.remove();  
    this.map = undefined;  
  }  
}
```

También se destruye el mapa al salir de la página (ionViewWillLeave()), evitando fugas de memoria y errores de renderizado al volver a entrar.

```
ionViewWillLeave() {  
  if (this.map) {  
    this.map.remove();  
    this.map = undefined;  
  }  
}
```

Página Consultar Fichaje

Esta página permite consultar los fichajes ya registrados, filtrando por usuario y fecha.

El usuario selecciona un nombre y una fecha, y al pulsar “Consultar Fichajes” se llama al método `consultarFichajes()`, que formatea la fecha y hace la petición al endpoint `GET /fichajes/usuario`.

```
consultarFichajes() {  
  if (!this.usuarioSeleccionado || !this.fechaInicio) {  
    alert('Debe seleccionar usuario y una fecha.');    return;  
  }  
  
  // Convierte la fecha seleccionada a formato YYYY-MM-DD  
  const fechaFormateada = this.fechaInicio.split('T')[0];  
  
  this.fichajesService.getFichajesByUsuario(this.usuarioSeleccionado, fechaFormateada)  
    .subscribe({  
      next: res => {  
        this.fichajes = res;  
        this.consultaRealizada = true;  
      },  
      error: err => {  
        console.error('Error consultando fichajes', err);  
        this.fichajes = [];  
        this.consultaRealizada = true;  
      }  
    });  
}
```

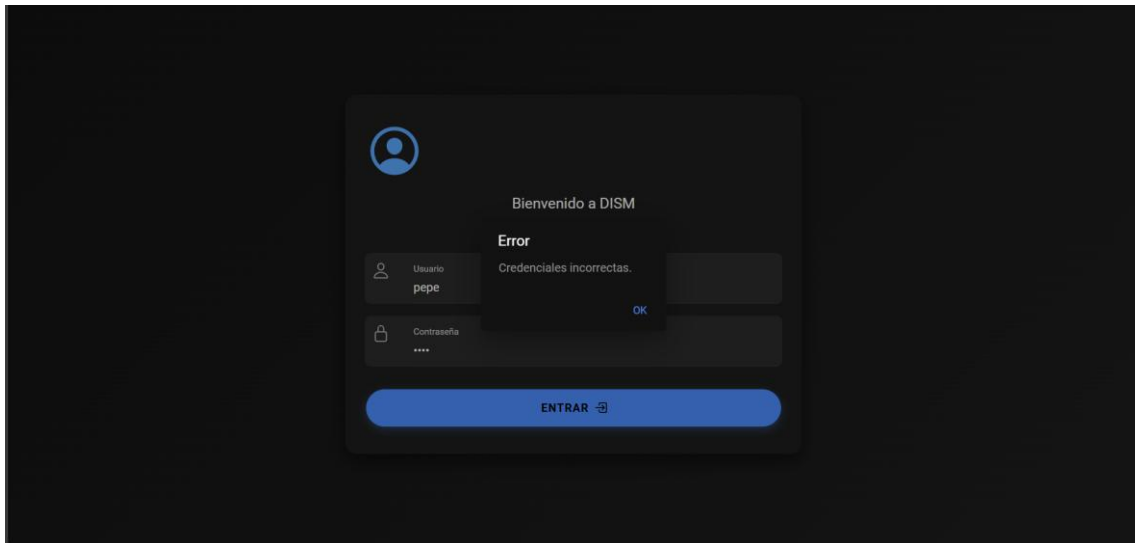
Los resultados se muestran en tarjetas con los datos del trabajo, hora de entrada, hora de salida, horas trabajadas y coordenadas. Si no se encuentran resultados, se muestra un mensaje informativo.

Aplicación AppAdmin

Para esta entrega he desarrollado una nueva aplicación independiente encargada de la gestión del sistema.

Acceso Restringido

Aunque el login es similar al de usuarios, en AppAdmin he implementado un control de seguridad adicional. Tras validar las credenciales, el sistema comprueba si el usuario es "Admin". Si intento acceder con un usuario normal, la aplicación denegará el acceso mediante una alerta.



Gestión de Trabajos y Usuarios

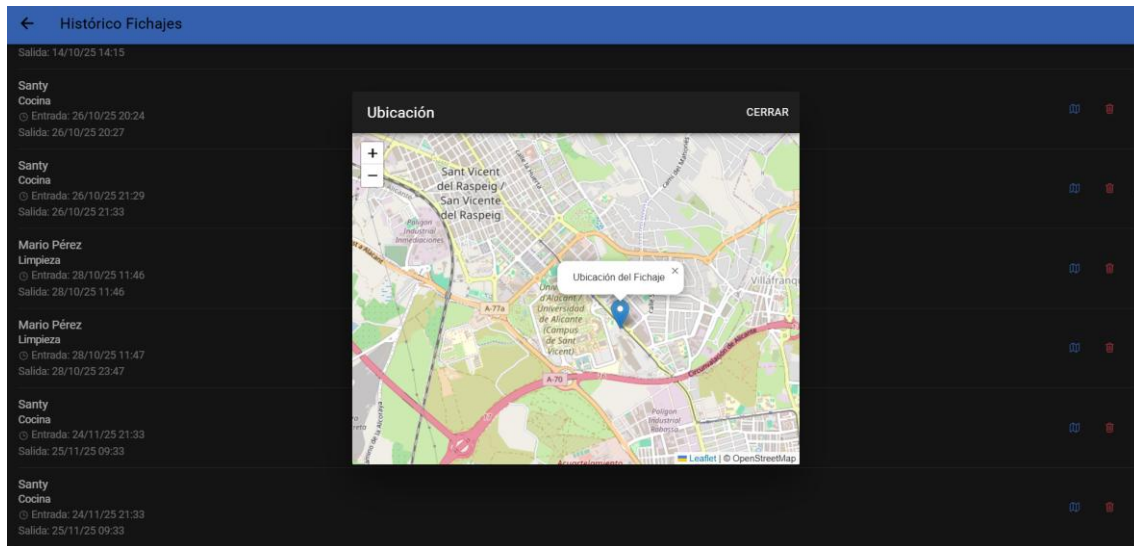
En estos apartados puedo gestionar los recursos de la empresa. He dispuesto un listado con todos los registros y, mediante un botón, se abre un formulario modal que permite:

- Crear nuevos usuarios o trabajos.
- Editar la información existente.
- Eliminar registros de la base de datos.

← Gestión Usuarios		+	
Mario Pérez	User: Mario99	✍	🗑
Santy	User: CamSan	✍	🗑
pru	User: pru	✍	🗑
Administrador del Sistema	User: Admin	✍	🗑

Gestión de Fichajes y Mapas

Este es el apartado más completo de la administración. Aquí puedo consultar el histórico de fichajes filtrando por un usuario concreto y por un rango de fechas. Al obtener los resultados, visualizo la información de entrada y salida. Además, si pulso en el icono del mapa de un fichaje, se abre un visor interactivo (Leaflet) que indica la posición geográfica exacta (latitud/longitud) donde se realizó dicho fichaje.



Seguridad y Autenticación

Para cumplir con los requisitos de la entrega y los apartados optativos, he realizado modificaciones importantes en el servidor NodeJS.

Middleware de Seguridad (ApiKey)

En este apartado he asegurado el acceso a mi API. Ahora, el servidor intercepta las peticiones y verifica que contengan una cabecera x-api-key válida.

- El sistema consulta la tabla ApiKey de la base de datos.
- Si la clave es correcta, permito el acceso a los endpoints (Usuarios, Fichajes, etc.).
- Si no es válida, devuelvo un error de autorización, protegiendo así la información.

```
const connection = require('../config/database');

const checkApiKey = (req, res, next) => {
  // Extraer clave de API del header
  const apiKey = req.headers['x-api-key'];

  if (!apiKey) {
    return res.status(401).json({ error: 'Acceso denegado. Falta la API Key' });
  }

  // Validar existencia de la clave en base de datos
  const query = 'SELECT * FROM apikey WHERE `Key` = ?';

  connection.query(query, [apiKey], (err, results) => {
    if (err) {
      return res.status(500).json({ error: 'Error al validar la API Key' });
    }

    if (results.length > 0) {
      // Permitir acceso con clave válida
      next();
    } else {
      res.status(403).json({ error: 'API Key no válida.' });
    }
  });
};

module.exports = checkApiKey;
```

Autenticación (Login)

He habilitado el endpoint /usuarios/login, el cual permite validar las credenciales (Usuario y Clave) recibidas desde las Apps. Al realizar esta petición, obtengo el objeto completo del usuario, lo que me sirve para identificar quién está accediendo y qué permisos tiene.

Swagger (OpenApi)

Swagger se ha utilizado para documentar de manera estructurada y visual todos los endpoints del Servicio REST del proyecto *Sistema de Registro Horario DISM*. La especificación está definida en formato OpenAPI 3.0.0 dentro del archivo `openapi.yaml`, ubicado en la carpeta raíz del proyecto `ServiciosRest`.

Este archivo describe todos los recursos disponibles en la API (Usuarios, Trabajos, Fichajes y ApiKeys), incluyendo los métodos HTTP, parámetros, cuerpos de petición y esquemas de respuesta.

Definición de Metadatos

```
openapi: 3.0.0
info:
  contact:
    email: info@dtic.org
  description: API para sistema de registro horario - Práctica 1 DISM
  license:
    name: Apache 2.0
    url: http://www.apache.org/licenses/LICENSE-2.0.html
  title: Sistema de Registro Horario DISM
  version: 1.0.0
servers:
  - description: Servidor ApiRest DTIC
    url: http://localhost:8080
```

Ejemplo Método HTTP (GET Usuarios)

```
/usuarios:
  get:
    Scan | Try it | Audit
    summary: Obtener todos los usuarios
    operationId: getUsers
    responses:
      '200':
        description: Lista de usuarios
        content:
          application/json:
            schema:
              type: array
              items:
                $ref: '#/components/schemas/Usuario'
      '500':
        description: Error del servidor
    tags:
      - Usuarios
    x-eov-operation-handler: controllers/UsuariosController
```

Components / Schema (Usuario)

```
schemas:
  Usuario:
    type: object
    properties:
      IdUsuario:
        type: integer
      Nombre:
        type: string
      Usuario:
        type: string
      Clave:
        type: string
```

Interfaz Visual del Swagger

Al ejecutar el proyecto con npm start, el servidor se lanza en el puerto 8080 y la interfaz de Swagger se puede abrir desde cualquier navegador en:

<http://localhost:8080/api-docs/>

Desde esta interfaz se pueden:

- Consultar todos los endpoints clasificados por grupo (Usuarios, Trabajos, Fichajes)
- Ver el formato exacto de las peticiones y respuestas
- Probar las operaciones directamente usando el botón “Try it out”, enviando datos reales y viendo la respuesta del servidor.

Manual de Usuario

El sistema desarrollado para la Práctica 1 está compuesto por dos partes principales: por un lado, el módulo ServiciosRest, encargado de ofrecer la API que gestiona los datos de usuarios, trabajos, fichajes y claves de acceso; y por otro, la aplicación AppUsuarios, que sirve como interfaz visual para que los usuarios puedan realizar y consultar sus fichajes desde un entorno Ionic.

Ambos módulos se encuentran dentro de la carpeta general del proyecto, llamada Practica1.

Puesta en Marcha ServicioRest

Para comenzar es primordial iniciar el servidor MySQL, en mi caso lo he hecho con XAMPP. Posteriormente iremos al directorio en el que se encuentra el servicio: Practica1/ServiciosRest e iniciaremos el proyecto mediante npm start, obteniendo el siguiente resultado en la consola.

```
PS C:\Proyectos\Practical\ServiciosRest> npm start

> servicios-rest-dism@1.0.0 prestart
> npm install

up to date, audited 447 packages in 2s

98 packages are looking for funding
  run `npm fund` for details

9 vulnerabilities (5 low, 1 high, 3 critical)

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.

> servicios-rest-dism@1.0.0 start
> node index.js

Listening on port 8080
=====
Servicios REST de DISM escuchando en el puerto 8080
Documentación Swagger disponible en: http://localhost:8080/api-docs/
=====
{"level":"info","message":"Servicios REST DISM en ejecución correctamente","service":"servicios-rest-dism","swagger":"http://localhost:8080/api-docs/","timestamp":"2025-10-25T21:23:43.202Z"}
info: Servicios REST DISM en ejecución correctamente {"service":"servicios-rest-dism","swagger":"http://localhost:8080/api-docs/","timestamp":"2025-10-25T21:23:43.202Z"}
```

Tras iniciarlo, estará disponible en <http://localhost:8080/api-docs/> la interfaz de Swagger para probar todos los endpoints del sistema.

Ejecución de la aplicación Ionic

Con el backend en funcionamiento, procederemos a iniciar la aplicación Ionic. Sin cerrar la terminal del ServicioRest, abriremos una terminal nueva en el directorio de AppUsuarios y AppAdmin. Para arrancarlo usaremos el comando ionic serve.

```
PS C:\Proyectos\Practical\AppUsuarios> ionic serve
> ng.cmd run app:serve --host=localhost --port=8100
[ng] > Building...
[ng] ✓ Building...
[ng] Initial chunk files | Names | Raw size
[ng] styles.css | styles | 70.13 kB |
[ng] main.js | main | 3.13 kB |
[ng] chunk-R7WCV5R6.js | - | 793 bytes |
[ng] polyfills.js | polyfills | 143 bytes |
[ng] | Initial total | 74.20 kB
[ng] Lazy chunk files | Names | Raw size
[ng] registrar-fichaje.page-NXYC5G5V.js | registrar-fichaje-page | 21.57 kB |
[ng] consulta-fichajes.page-P6MP5PLT.js | consulta-fichajes-page | 12.87 kB |
[ng] home.page-LWGQWQ5T.js | home-page | 6.93 kB |
[ng] chunk-QHL4XN4D.js | - | 2.96 kB |
[ng] Application bundle generation complete. [3.964 seconds] - 2025-10-25T21:29:46.854Z
[ng] NOTE: Raw file sizes do not reflect development server per-request transformations.

[INFO] Development server running!

Local: http://localhost:8100

Use Ctrl+C to quit this process

[ng] → Local: http://localhost:8100/
[ng] → press h + enter to show help
[INFO] Browser window opened to http://localhost:8100!
```

Automáticamente se abrirá la web con la AppUsuarios y otra con AppAdmin en funcionamiento y por último activaremos la ubicación de nuestro pc.

Pruebas del Funcionamiento

Tras seguir los pasos del manual de usuario y tener el proyecto completo en marcha, ya podemos utilizar la aplicación.

AppUsuarios

Registro de un Fichaje

Elegiremos el usuario y el trabajo (ya creados en la base de datos) y obtendremos la geolocalización.

Registrar Fichaje

Usuario: Mario Pérez ▼

Trabajo: Limpieza ▼

OBTENER GEOLOCALIZACIÓN

Latitud: [Placeholder]

Longitud: [Placeholder]

Dirección: [Placeholder]

Comunidad: [Placeholder]

[Map showing Sant Blai area]

INICIAR FICHAJE

La geolocalización funciona perfectamente, pero por privacidad he decidido tapar los datos.

Tras pulsar el botón de “iniciar fichaje”, se registrará en el sistema y se actualizará la página mostrando el fichaje activo.

Registrar Fichaje

Fichaje Activo

Trabajo: Limpieza

Entrada: 10/26/25, 10:29 AM

FINALIZAR FICHAJE

Si finalizamos el fichaje la página se actualizará y permitirá registrar una nuevo.

En la terminal durante el proceso veremos líneas como las siguientes:

```
Listening on port 8080
=====
Servicios REST de DISM escuchando en el puerto 8080
Documentación Swagger disponible en: http://localhost:8080/api-docs/
=====
{"level":"info","message":"Servicios REST DISM en ejecución correctamente","service":"servicios-rest-dism","swagger":"http://localhost:8080/api-docs/","timestamp":"2025-10-26T09:20:30.703Z"}
info: Servicios REST DISM en ejecución correctamente {"service":"servicios-rest-dism","swagger":"http://localhost:8080/api-docs/","timestamp":"2025-10-26T09:20:30.703Z"}
GET /usuarios llamado
GET /trabajos llamado
Conectado a la BD MySQL
GET /fichajes/actual/2 llamado
POST /fichajes llamado con: {
  IdUsuario: 2,
  IdTrabajo: 2,
  GeolocalizacionLatitud: [REDACTED],
  GeolocalizacionLongitud: [REDACTED]
}
GET /fichajes/actual/2 llamado
PUT /fichajes/14/finalizar llamado
```

Consulta de Fichaje

En este apartado podremos obtener los fichajes de un usuario con fecha de inicio elegida por nosotros.

Tras elegir el usuario y la fecha de inicio del fichaje, obtendremos la información relacionada con el mismo. En este caso, las horas trabajadas figuran como 0 ya que únicamente han pasado 4min desde el inicio y fin del fichaje.

Consulta de Fichajes

Usuario

Mario Pérez ▾

Fecha

octubre de 2025 ▾

< >

D	L	M	X	J	V	S
			1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30	31	

CONSULTAR FICHAJES

Limpieza

Entrada: 10/26/25, 10:29 AM
Salida: 10/26/25, 10:33 AM
Horas trabajadas: -
Latitud:
Longitud:

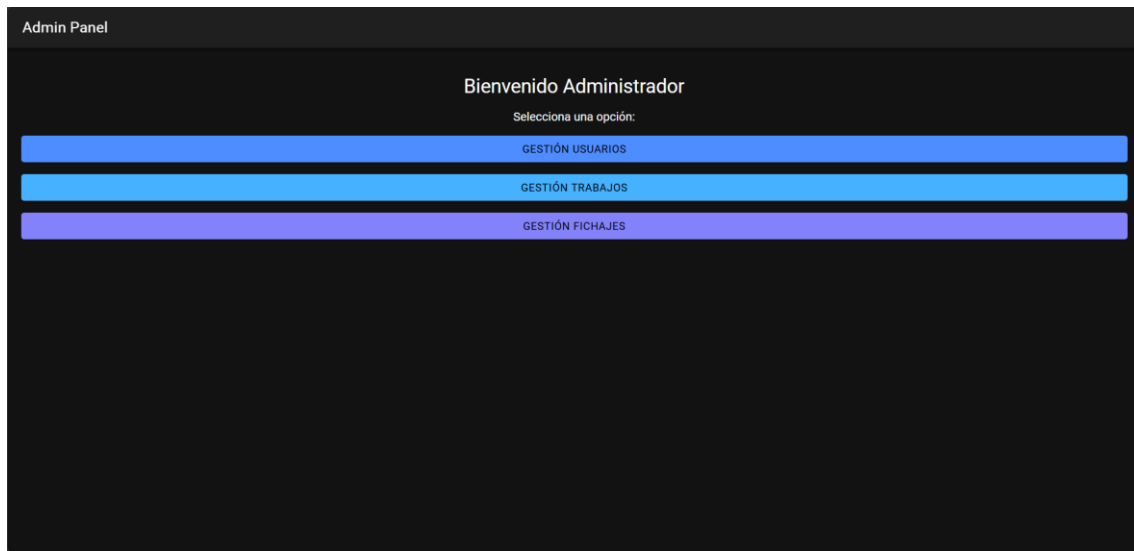
```
GET /fichajes/usuario llamado con: { usuarioId: 2, fechaInicio: '2025-10-26' }
```

En la siguiente captura del propio PHPMyAdmin se puede ver el correcto funcionamiento del cálculo de horas trabajadas con otro fichaje.

IdFichaje	FechaHoraEntrada	FechaHoraSalida	HorasTrabajadas	IdTrabajo	IdUsuario	GeolocalizacionLatitud	GeolocalizacionLongitud
3	2025-10-11 09:15:00	2025-10-11 17:22:00	8	2	2		

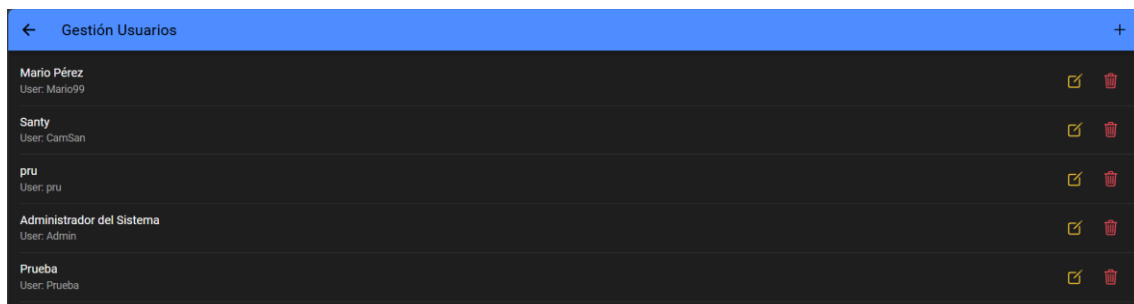
AppAdmin

Pantalla Inicial tras Login

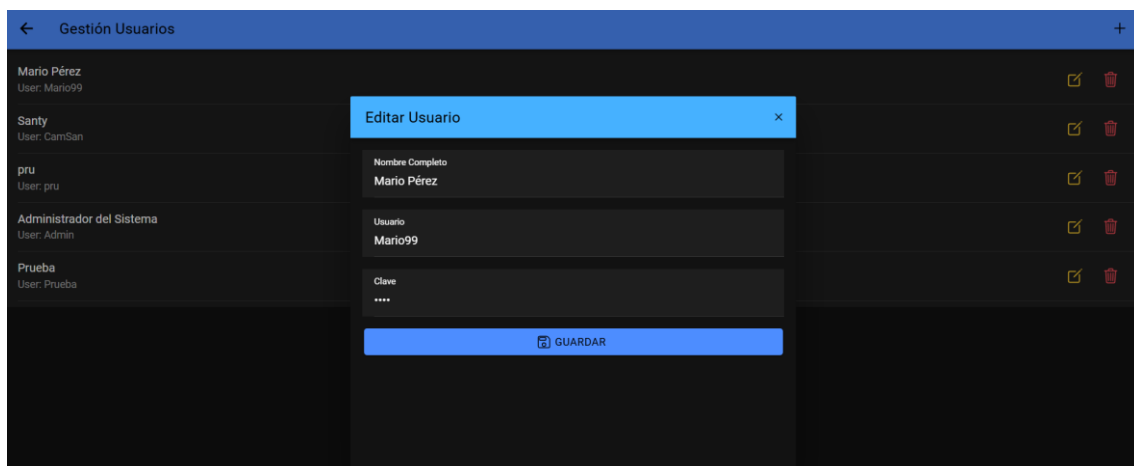


Gestión de Usuarios

Listado



Edición



Gestión de Trabajos

Listado

← Gestión Trabajos +		
	Limpieza ID: 2	 
	Cocina ID: 5	 
	Prueba1 ID: 8	 

Edición

← Gestión Trabajos +

 Limpieza
ID: 2

 Cocina
ID: 5

 Prueba1
ID: 8

Editar Trabajo ×

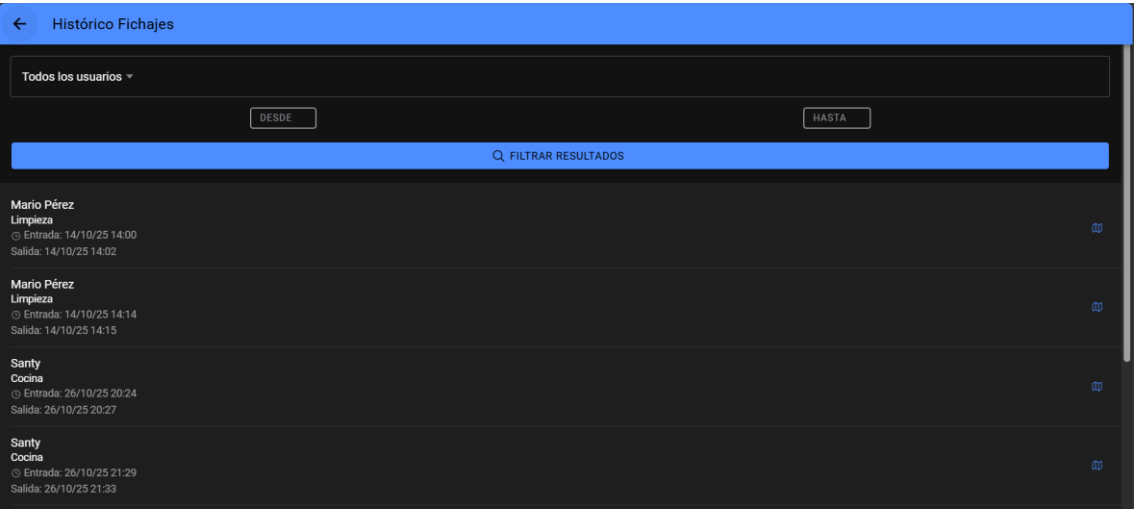
Nombre del Trabajo

Limpieza

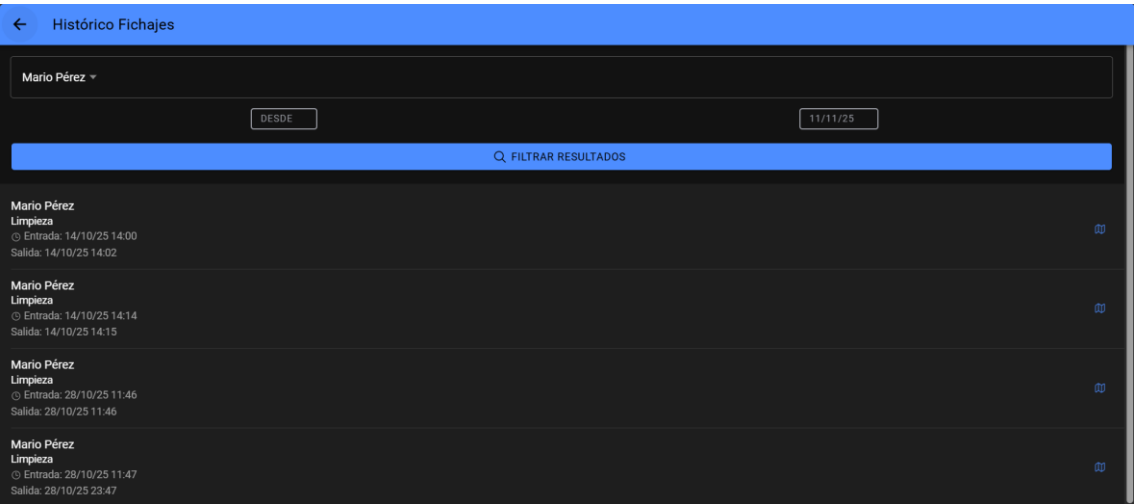
GUARDAR

Gestión de Fichajes

Listado



Filtrado por Fecha y Usuario



Visualización Mapa

